session 11 vectors and matrices(Basics)

1D Vectors, 2D Matrices, Matrix Operations & Tabular Computation in Python

Task 1: Playlist Song Durations (1D Vector Sum)

Scenario: Represent a music playlist of 5 song durations in seconds as a 1D vector and calculate its total runtime.

```
# 1D Vector representing 5 song durations (in seconds) playlist_vector = [210, 180, 245, 200, 175] # Calculate total playlist duration total_duration_seconds = sum(playlist_vector) minutes = total_duration_seconds // 60 seconds = total_duration_seconds % 60 print(f"Playlist Vector: {playlist_vector}") print(f"Total Duration: {total_duration_seconds} seconds ({minutes}m {seconds}s)")
```

PROGRAM OUTPUT

Playlist Vector: [210, 180, 245, 200, 175]
Total Duration: 1010 seconds (16m 50s)

Task 2: Swiggy Food Ratings (3x3 Matrix Slicing)

Scenario: Store ratings (1–5) given by 3 friends across 3 food items in a 3x3 matrix and display the ratings for the second food item (column index 1).

```
# 3x3 Matrix: Rows = Friends (Friend 1, 2, 3), Cols = Items (Item 1, Item 2, Item 3) ratings_matrix = [ [5, 4, 3], # Friend 1 ratings [3, 5, 2], # Friend 2 ratings [4, 4, 5] # Friend 3 ratings ] print("3x3 Swiggy Ratings Matrix:") for row in ratings_matrix: print(row) print("\nRatings for Second Food Item (Item 2):") for i, friend_ratings in enumerate(ratings_matrix, start=1): print(f"Friend {i}: Rating = {friend_ratings[1]}")
```

MATRIX INDEXING OUTPUT

Matrix: [[5, 4, 3], [3, 5, 2], [4, 4, 5]]
- Friend 1 rated Item 2: **4 / 5**
- Friend 2 rated Item 2: **5 / 5**
- Friend 3 rated Item 2: **4 / 5**

Task 3: Daily Restaurant Dish Orders (Column Summing)

Scenario: A 3x4 matrix stores order counts for 4 dishes across 3 days (rows: days, columns: dishes). Compute the total orders for each dish.

```
# Rows = 3 Days, Columns = 4 Dishes [Biryani, Pizza, Burger, Pasta] orders_matrix = [ [15, 20, 12, 8], # Day 1 [18, 15, 10, 12], # Day 2 [22, 25, 15, 10] # Day 3 ] dish_names = ["Biryani", "Pizza", "Burger", "Pasta"] # Summing across columns for each dish num_dishes = len(orders_matrix[0]) dish_totals = [sum(orders_matrix[day][dish] for day in range(len(orders_matrix))) for dish in range(num_dishes)] for name, total in zip(dish_names, dish_totals): print(f"Total Orders for {name}: {total}")
```

COLUMN TOTALS RESULT

Biryani: **55 orders** | Pizza: **60 orders** | Burger: **37 orders** | Pasta: **30 orders**

Task 4: Flipkart Revenue Calculation (Matrix Row Multiplication)

Scenario: Given a 5x2 matrix of Flipkart products (Column 0: Price in ₹, Column 1: Quantity Sold), write a function to calculate total revenue per product.

```
def calculate_product_revenues(product_matrix): revenues = [] for idx, row in enumerate(product_matrix, start=1): price = row[0] qty = row[1] rev = price * qty revenues.append(rev) print(f"Product {idx} (Price: ₹{price}, Qty: {qty}): Revenue = ₹{rev:,}") return revenues # 5x2 Matrix: Rows = Products, Col 0 = Price, Col 1 = Quantity flipkart_matrix = [ [15000, 10], # Product 1 [800, 50], # Product 2 [2500, 20], # Product 3 [45000, 5], # Product 4 [1200, 30] # Product 5 ] print("--- Flipkart Revenue Computation ---") total_revs = calculate_product_revenues(flipkart_matrix) print(f"\nTotal Gross Revenue Across All Products: ₹{sum(total_revs):,}")
```

REVENUE OUTPUT SUMMARY

- Product 1: ₹150,000 | Product 2: ₹40,000 | Product 3: ₹50,000
- Product 4: ₹225,000 | Product 5: ₹36,000
- **Gross Platform Revenue: ₹501,000**